



QwenPI VLA Training Optimization

NVIDIA DevTech

Results at a Glance

starVLA QwenPI: Qwen3.5-VL 4B + flow-matching action head, ZeRO-2 full fine-tune

3.2×

Faster training step

800 → 249 ms/step (bs 8)

257

Samples / node / s

up from 80 at hand-off

23.1%

**Model FLOPs
utilization**

up from 7.2% (bf16 peak)

340

Samples / node / s

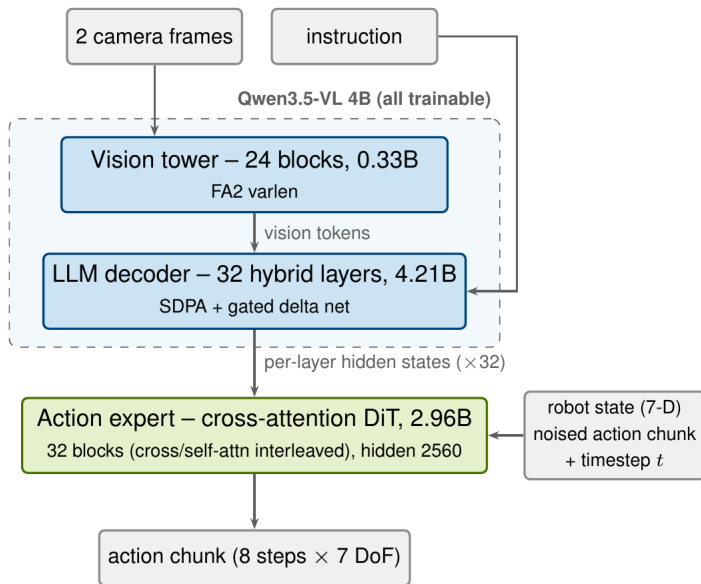
at bs 24 — 1.7× the 200
target

Method: every optimization was profiled with Nsight Systems and validated with a same-node A/B run plus loss-trajectory checks — no stacked guesses, measured deltas only.

Measurement: 8×H200 (NVLink), 60 steps, steady-state p50 of steps 21–44, per-GPU batch 8 unless noted.

Model & Workload

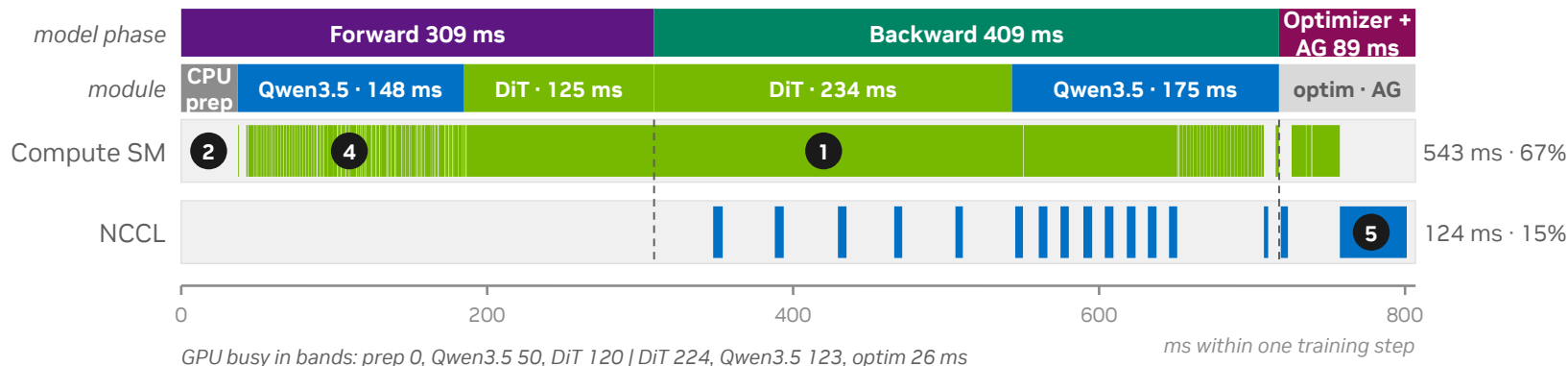
7.5B trainable parameters: Qwen3.5-VL 4B backbone + 3B flow-matching action expert



Training configuration	
Parameters	7.52B, all trainable (bf16)
Parallelism	DeepSpeed ZeRO-2, DP=8, 1 node
ZeRO-2 tuning	bucket 1.5e9, reduce_scatter off
Optimizer	AdamW, fp32 master (sharded)
Batch	8 / GPU $\rightarrow$ 64 global (up to 24/GPU)
Sequence	text fixed at 192 (left-pad)
Data	LIBERO manipulation, 2 views/sample
Hardware	8 $\times$ H200 SXM, full NVLink
Software	torch 2.7.1 / NCCL 2.26.2 / HF 5.3 / FA2.8

Baseline Diagnosis: Where Did 800 ms Go?

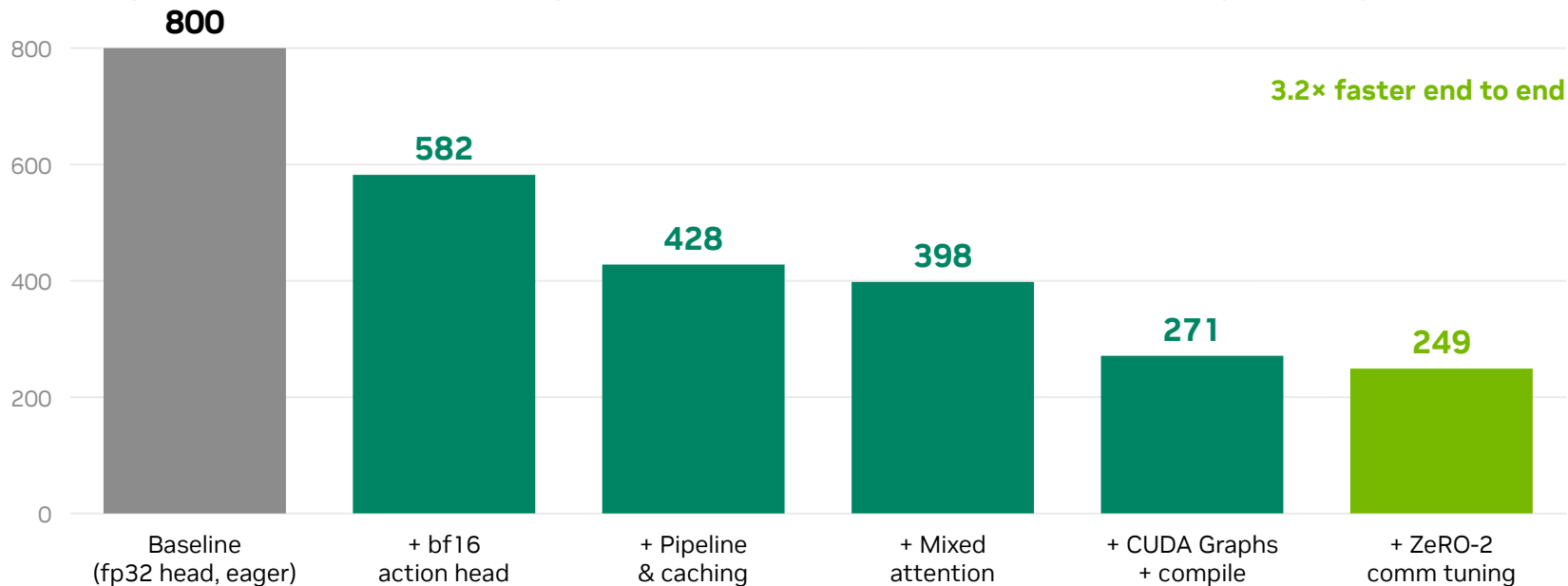
One baseline step on the GPU-execution clock (807 ms, bs 8/GPU) — five problems, five fixes



1 · fp32 action head	201 ms fp32 GEMMs vs 66 ms bf16 — 2.96B DiT outside autocast	bf16 DiT (−218 ms)
2 · CPU inside forward()	first ~40 ms of every step: compute idle, HF preprocessing on critical path	collate in DataLoader workers
3 · Per-step syncs	loss readback, DS timers, MRoPE position-ids, mm-merge check	sync-free logging + caches (2+3: −117)
4 · Launch-bound eager	13,868 launches @ 8 μs median; occupancy fwd 34% vs bwd 70%; idle 205 ms	CUDA Graphs + compile (−126 ms)
5 · Comm configuration	15 small AR buckets; 22 ms redundant flatten; exposed AllGather tail	bucket 1.5e9 + reduce_scatter off (−22)

Optimization Roadmap

Actual step time (ms) after each optimization lands — same node, steady-state p50



Cumulative levels re-measured per step with nsys attached on rank 0 (~5% overhead). The torch 2.7.1 + NCCL 2.26.2 environment upgrade landed between the bf16 and pipeline levels and is folded into the Pipeline bar. Final 249 ms = clean steady-state p50 of merged main.

Fix 1 · bf16 Action Head: -218 ms

800 → 582 ms. One config knob — and numerically a no-op for training

GPU busy by kernel class (baseline step, 543 ms)

201 ms

fp32 GEMM — all from the action head

126 ms

other (fla/GDN, embeddings, misc)

102 ms

elementwise / norm / reduce

66 ms

bf16 GEMM (Qwen3.5)

27 ms

copy / gather / cat

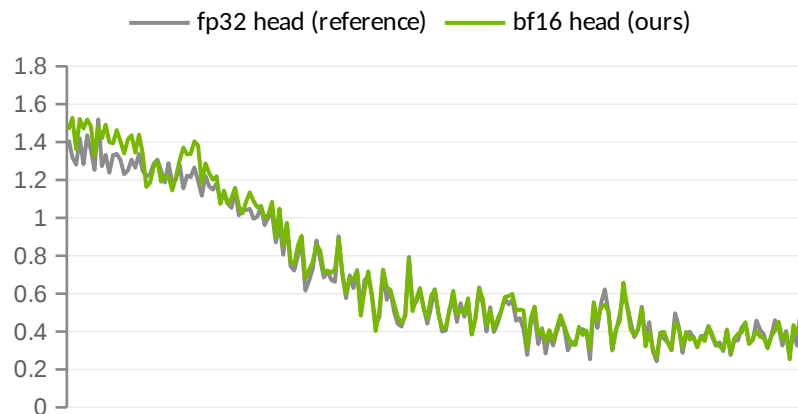
22 ms

attention kernels

37% of all GPU math is fp32 — all of it from the 2.96B DiT outside the bf16 autocast region.

Why was it fp32 at all? Inherited caution for diffusion numerics. Only the timestep embedding and the loss math need fp32 — we kept exactly those in fp32 and moved the transformer body to bf16 (dit_dtype knob, one-line rollback).

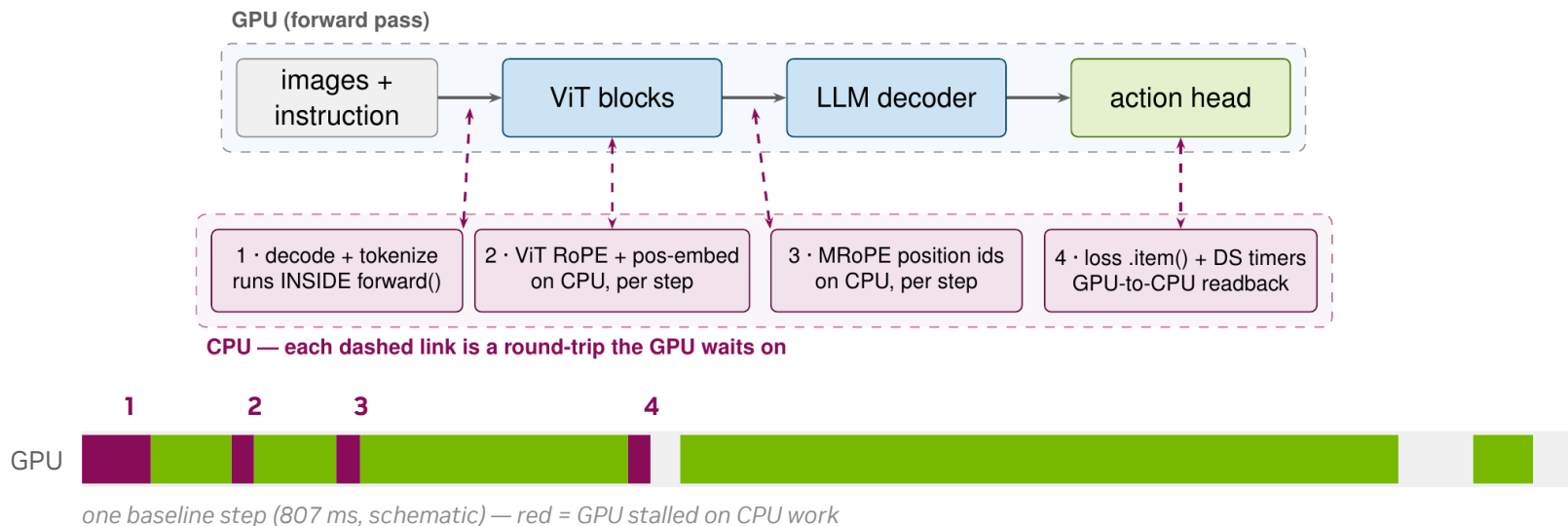
Same seed, 200 steps: loss is unchanged



action loss, training steps 1 → 200 · mean per-step $|\Delta| = 0.04$ (run-to-run noise floor) · final: 0.549 vs 0.533

Sync Points: Where the CPU Blocks the GPU

Four CPU round-trips hide inside every baseline training step

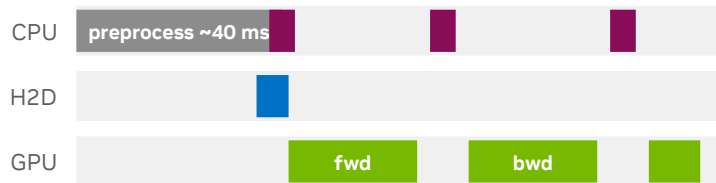


Measured: stall 1 $\approx$ 37 ms at every step start; stall 3 $\approx$ 13 ms; stall 4 drains the launch pipeline twice per step — together seeding the 205 ms/step of GPU idle. The next page removes all four.

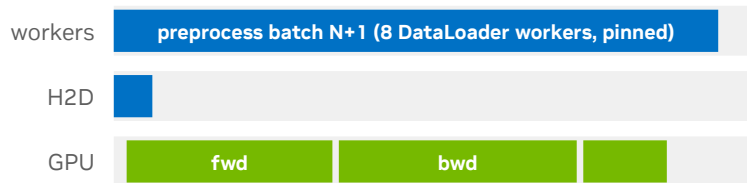
Removing the Sync Points: -117 ms

545 → 428 ms. Every CPU round-trip moves off the critical path

Before — everything serialized inside forward()



After — CPU work overlaps the previous step



red = per-step syncs (loss readback, DS timers, mm-merge check): each one drains the launch pipeline and strands the GPU

Preprocess in workers	HF processor (image patches + tokenize) runs in 8 workers, overlapped with the previous step	~40 ms off the critical path
Side-stream H2D	pinned host batch + dedicated copy stream, overlaps the ZeRO-2 optimizer tail	upload hidden
Sync-free logging	DeepSpeed timers unsynchronized; loss .item() only on logging steps	2 device syncs / step removed
MRoPE position-id cache	3D position ids precomputed on CPU, cached by image grid	~13 ms sync eliminated
+ Mixed attention	vision tower FA2 varlen (16 images / kernel); text stays SDPA — full FA2 measured slower at seq 192	-30 ms (428 → 398)

Why CUDA Graphs: Launch-Bound Forward

Anatomy of the Qwen3.5 forward — same math, very different wall clock

**Baseline
(eager)**



3,929 kernel launches · 19.1 ms of cudaLaunchKernel CPU time · occupancy 34%

**Optimized
(CUDA Graphs)**



6 graph launches + 736 eager · 9.4 ms launch CPU · occupancy 57%

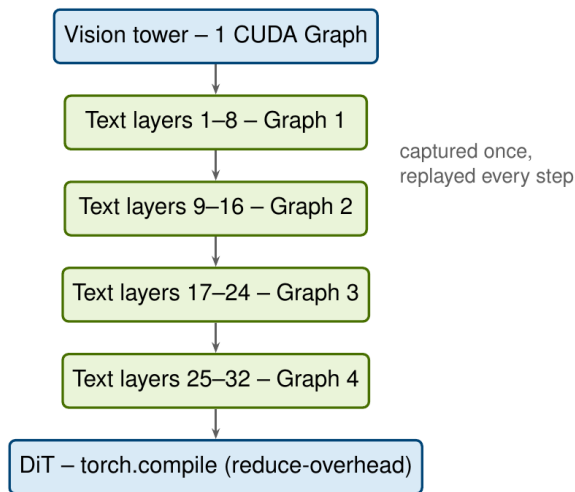
2.3× faster forward wall clock — with 81% fewer launches

- An eager op costs 30–60 μ s of Python dispatch + launch; the median kernel runs 8 μ s — the GPU finishes before the CPU can feed the next one
- Backward suffers far less (C++ autograd engine, occupancy 70%) — forward is where launch overhead bites
- A CUDA Graph replays an entire 8-layer group as ONE cudaGraphLaunch — per-op dispatch leaves the critical path

Busy also shrinks 50 → 37 ms (mixed attention + fusion); the split isolates the launch effect. Windows: baseline Qwen3.5 fwd vs merged-main VLM fwd, same launch-window accounting.

CUDA Graphs — and Why Four, Not One

398 → 271 ms (−126). Capture once, replay every step



Enablers: static shapes (pad-192) · vision tower = 1 graph · DiT = torch.compile · merge sync-free eager

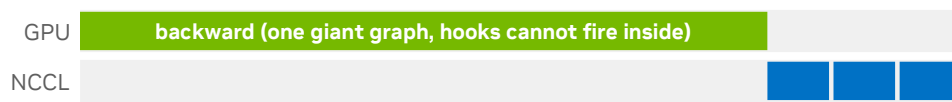
Why is the decoder FOUR graphs, not one?

DeepSpeed's compute-comm overlap must survive: AllReduce hooks fire only BETWEEN replays — never inside one.

4 graphs — AllReduce overlaps the group replays:



1 graph — every AllReduce waits for the whole backward:



same compute, same bytes — the red line is where the step ends

Kernel launches: 13.9k → 6.4k per step. CPU launch cost leaves the critical path; GPU idle collapses.

ZeRO-2 Comm Tuning: Two Knobs

271 → 249 ms. Two different problems: a redundant flatten, and a blocked backward

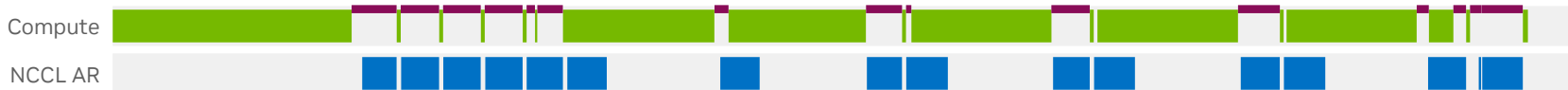
Knob 1 · `reduce_scatter` → `false` (−13 ms)

- Default path flattens every bucket (22 ms/step CatArrayBatchedCopy) — but grads already sit contiguous
- Plain AllReduce skips it. Cost: ~2× wire bytes — fine while comm overlaps and NVLink has headroom

Knob 2 · `reduce_bucket_size` → `1.5e9` (−12 ms)

- Grads double-buffer into TWO buckets; every boundary makes compute wait on the reduction stream
- Too small → backward stalls behind its own AllReduces (below). Too big → exposed tail (2e9 worse)

bucket 5e8 — backward 167 ms · 18 AllReduces · 15 stalls, 47 ms of blocked compute



bucket 1.5e9 — backward 154 ms · 8 AllReduces · 8 stalls, 27 ms

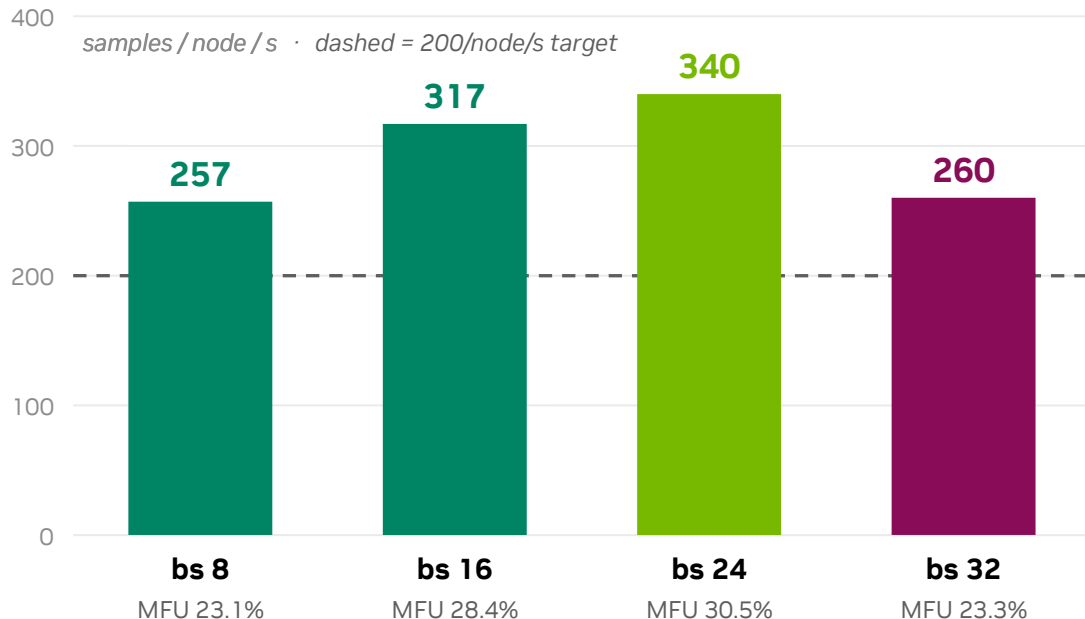


one backward pass, real kernels (node-level nsys, same node, same bytes) · red = compute stream stalled at a bucket rendezvous

No conflict — they compose: bucket size = how OFTEN compute and reduction rendezvous; `reduce_scatter` = what each rendezvous COSTS. `false` makes each AR bigger, strengthening the case for large buckets.

Batch Scaling: Sweet Spot at 24 per GPU

Same optimized config, only per-GPU batch changes — peak 340 samples/node/s



- Batching amortizes launch and comm costs: MFU climbs 23% → 30.5%
- bs 32 regresses hard (986 ms/step): not OOM, not the dataloader — memory-pressure effects under investigation
- Larger global batch (192) changes optimization dynamics — needs a convergence run before adoption

What We Ruled Out — and How

Negative results are results: every hypothesis got a same-node experiment

Hypothesis	Experiment	Verdict
FA2 for the text stack	A/B at seq 192	Slower (+10 ms): unpad/repad overhead exceeds kernel gains at short sequences
“NCCL is misconfigured”	Forced protocols, NCCL_DEBUG, microbenchmark	AllReduce already at 478 GB/s busbw = NVLink ceiling; kernel-name suffix was a red herring
masked_scatter nonzero sync	index_copy patch, bitwise-equal unit test	0 ms: the AllReduce tail is gradient-readiness-bound, so hiding the sync moves nothing
MRoPE cache still needed?	Toggle at final config	Neutral at the tip (CUDA Graphs hide it) — kept for CPU headroom, one-knob rollback
Whole-model torch.compile	Compile attempts on torch 2.6 & 2.7	Hybrid GDN layers graph-break in dynamo → group-graph capture instead
FlashQLA GDN kernels	Kernel-time ceiling analysis	Only ~10.5 ms/step of GDN time exists; needs torch $\geq$ 2.8 — deferred, not worth the risk now

Remaining Headroom & Next Steps

Where the next milliseconds live, ranked by ceiling — measured, not guessed



Backward GEMMs at the power wall [fp8 territory]

Sustained GEMM throughput caps at ~622 TFLOPS on this node (power, not occupancy). The only lever left for the math itself is fp8.



Exposed communication (~30-55 ms) [compression]

Bandwidth is at the NVLink ceiling; bytes are the only lever — 1-bit Adam / gradient compression, with convergence sign-off.



Exposed parameter AllGather (44 ms) [FSDP or async ZeRO-2]

Optimization shrank compute — the step-end AllGather is now fully exposed and ZeRO-2 cannot overlap it. Adopt FSDP (overlapped unshard) or extend ZeRO-2 with an async parameter gather.



Scale-out readiness [deployment]

On split-topology nodes (4+4 NVLink islands) set NCCL_P2P_LEVEL=NVL — first collective hangs otherwise. bs24 adoption gated on a convergence run.